

Exame de Paradigmas de Programação — Época de Recurso / Modelo 3

|||

|---|---|

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso / Modelo 3
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1

1,5 val

Explique o ciclo de vida dos objetos em Java e o papel do *Garbage Collector* (GC) na gestão automática de memória. Distinga a memória *Heap* da memória *Stack*, indicando o que é armazenado em cada uma e quais as vantagens de não haver desalocação manual de memória como em C/C++.

Pergunta 2

1,5 val

A partir do Java 8, as interfaces passaram a permitir a inclusão de métodos `default` e `static`. Explique a razão teórica e prática para a introdução destes métodos. Como diferem os métodos `default` de métodos abstratos e de métodos pertencentes a classes abstratas?

Pergunta 3

1,5 val

Distinga detalhadamente as relações de **Associação**, **Agregação** e **Composição** entre classes no paradigma de Orientação a Objetos. Apresente a diferença conceptual ao nível do ciclo de vida dos objetos envolvidos e ilustre cada relação com um pequeno exemplo de código Java.

Pergunta 4

1,5 val

Explique o contrato existente em Java entre os métodos `hashCode()` e `equals()` definidos na classe `Object`. Por que razão é estritamente obrigatório redefinir o método `hashCode()` sempre que o método `equals()` é redefinido numa classe? Quais as consequências de violar este contrato ao utilizar coleções ou estruturas de dados de pesquisa?

Parte 2

1. Considere as seguintes interfaces `Vehicle` e `ElectricVehicle`. A interface `Vehicle` define as propriedades genéricas de um veículo de recolha. A interface `ElectricVehicle` especializa `Vehicle` adicionando comportamentos específicos para veículos elétricos de emissão zero com gestão de bateria.

```
public interface Vehicle {  
    String getCode();  
    ItemType getSupplyType();  
    double getMaxCapacity();  
}
```

```
public interface ElectricVehicle extends Vehicle {  
    double getBatteryCapacity(); // em kWh  
    double getCurrentBattery(); // em kWh  
    boolean charge(double amount);  
    double getEstimatedAutonomy(); // em km  
    boolean equals(Object obj);  
}
```

Pergunta 1a

3 val

Implemente a interface `ElectricVehicle` numa classe denominada `ElectricVehicleImpl`.

Regras de Implementação:

- A classe deve manter a autonomia estimada (`getEstimatedAutonomy()`), assumindo um consumo fixo de `0.2 kWh` por quilómetro (ou seja, `autonomia = bateriaAtual / 0.2`).
- O método `charge(double amount)` deve adicionar a quantidade fornecida à bateria atual (`getCurrentBattery()`), sem contudo exceder a capacidade máxima (`getBatteryCapacity()`). Retorna `true` se o carregamento for realizado com sucesso.
- O método `equals` deve considerar que duas instâncias de `ElectricVehicle` são iguais se possuírem o mesmo código de veículo (`getCode()`).

Pergunta 1b

2 val

Desenvolva uma classe de teste com o método `main` para validar a classe `ElectricVehicleImpl`. O teste deve simular o carregamento do veículo, a verificação da autonomia estimada antes e depois do carregamento, e a comparação de dois veículos através do método `equals`.

2. Considere a existência de uma classe `RoutePlannerImpl` que implementa a interface `RoutePlanner`.

```
public interface RoutePlanner {  
    double calculateRouteTotalDistance(Route route, DistanceMatrix matrix);  
    boolean isRouteFeasible(ElectricVehicle vehicle, Route route, DistanceMatrix matrix);  
    Route[] filterFeasibleRoutes(ElectricVehicle vehicle, Route[] candidateRoutes,  
    DistanceMatrix matrix);  
}
```

Pergunta 2a

4 val

Na classe `RoutePlannerImpl`, implemente os seguintes métodos auxiliares:

```
double calculateRouteTotalDistance(Route route, DistanceMatrix matrix);
```

- Este método calcula a distância total (em km) para percorrer a rota.
- A distância total é a soma: da **Base** para a primeira `AidBox` da rota + entre cada `AidBox` consecutiva da rota + da última `AidBox` de regresso à **Base**.
- Utilize `matrix.getDistance(boxA, boxB)` para obter as distâncias entre caixas, e `matrix.getDistanceFromBase(box)` / `matrix.getDistanceToBase(box)` para os percursos com a Base.

```
boolean isRouteFeasible(ElectricVehicle vehicle, Route route, DistanceMatrix matrix);
```

- Este método determina se uma rota pode ser percorrida em segurança pelo veículo elétrico.
- A rota é viável se a energia necessária para percorrer a distância total (distância em km * 0.2 kWh/km) for menor ou igual à bateria atual do veículo (`vehicle.getCurrentBattery()`) E se a carga total acumulada dos contentores da rota não exceder a capacidade máxima do veículo (`vehicle.getMaxCapacity()`).

Pergunta 2b

5 val

Na classe `RoutePlannerImpl`, implemente o método `filterFeasibleRoutes`:

```
Route[] filterFeasibleRoutes(ElectricVehicle vehicle, Route[] candidateRoutes, DistanceMatrix  
matrix);
```

Regras a considerar:

- Percorra o array `candidateRoutes` fornecido.

- Utilize o método `isRouteFeasible` desenvolvido na alínea anterior para filtrar apenas as rotas que o veículo elétrico consegue realizar.
- O método deve devolver um novo array de `Route` perfeitamente dimensionado, contendo **apenas** as rotas viáveis (sem posições nulas ou rotas inviáveis).

Excertos de Código Fornecidos

```
public interface Route {
    AidBox[] getAidBoxes();
    double getTotalWeight();
}

public interface DistanceMatrix {
    double getDistance(AidBox from, AidBox to);
    double getDistanceFromBase(AidBox to);
    double getDistanceToBase(AidBox from);
}

public interface AidBox {
    String getCode();
}
```